ExaMPI: A Modern Design and Implementation to Accelerate Message Passing Interface Innovation

Anthony Skjellum[1(✉)], Martin Rüfenacht[1,3], Nawrin Sultana[2], Derek Schafer[4], Ignacio Laguna[5], and Kathryn Mohror[5]

[1] University of Tennessee at Chattanooga, Chattanooga, USA
{tony-skjellum,martin-ruefenacht}@utc.edu
[2] Auburn University, Auburn, USA
nzs0034@auburn.edu
[3] EPCC, University of Edinburgh, Edinburgh, Scotland, UK
[4] Tennessee Tech University, Cookeville, USA
djschafer42@students.tntech.edu
[5] Lawrence Livermore National Laboratory, Livermore, USA
{lagunaperalt1,kmohror}@llnl.gov

Abstract. The difficulty of deep experimentation with Message Passing Interface (MPI) implementations—which are quite large and complex—substantially raises the cost and complexity of proof-of-concept activities and limits the community of potential contributors to new and better MPI features and implementations alike. Our goal is to enable researchers to experiment rapidly and easily with new concepts, algorithms, and internal protocols for MPI, we introduce ExaMPI, a modern MPI-3.x subset with a robust MPI-4.x roadmap. We discuss design, early implementation, and ongoing utilization in parallel programming research, plus specific research activities enabled by ExaMPI.

Architecturally, ExaMPI is a C++17-based library designed for modularity, extensibility, and understandability. The code base supports both native C++ threading with thread-safe data structures and a modular progress engine. In addition, the transport abstraction implements UDP, TCP, OFED verbs, and LibFabrics for high-performance networks.

By enabling researchers with ExaMPI, we seek to accelerate innovations and increase the number of new experiments and experimenters, all while expanding MPI's applicability.

This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under contract DE-AC52-07NA27344. Lawrence Livermore National Security, LLC (LLNL-CONF-775497) and partial support from the National Science Foundation under Grants Nos. CCF-1562659, CCF-1562306, CCF-1617690, CCF-1822191, CCF-1821431. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation or Lawrence Livermore National Laboratory.

© Springer Nature Switzerland AG 2020
J. L. Crespo-Mariño and E. Meneses-Rojas (Eds.): CARLA 2019, CCIS 1087, pp. 153–169, 2020.
https://doi.org/10.1007/978-3-030-41005-6_11

Keywords: MPI · Middleware architecture · Parallel programming models · Performance portability · Cost of portability

1 Introduction

The complexity of leading open source implementations of MPI is daunting when it comes to experimentation and modification with new and different concepts for MPI-4 or other research experiments. Production open source MPIs have successfully focused on completeness of coverage, correctness, compliance, and, of course, middleware portability and performance. But, they typically leveraged software architectures rooted in legacy implementations of MPI-1 or earlier message passing systems, where assumptions were made based on then-extant architectures, processor resources, assumptions of intra-node concurrency, and performance levels. Production open source MPIs possess complex internal architectures, layers, and global state, and cross-cutting issues can arise when trying to experiment. Such issues make it difficult and expensive to achieve new changes, while also limiting certain kinds of experiments like overlapping of communication and computation.

To enhance and simplify researchers' ability to explore new and diverse functionality with MPI with quality performance potential, the authors have devised ExaMPI, a new, BSD-licensed open source implementation. A few factors enhance both the validity and necessity of building up a modular, research MPI at this time. First, over the past few years, a number of robust data movers such as Libfabric, Portals, and even InfiniBand verbs have decreased the importance of complex "channel devices" and other transport abstractions within MPI itself. These transports often include internal progress (independent of user calls to MPI) for sufficiently smart NICs and will soon include collective communication offload for some networks. Second, and as important, most MPI applications don't use a huge fraction of the MPI standard, which means that the complexity associated with full API support isn't needed for many kinds of applications and, hence, application experiments.

The remainder of this paper is organized as follows: Sect. 2 discusses selected prior MPI implementations. Section 3 offers motivations for ExaMPI. Section 4 discusses requirements. Section 5 explores the design of the internal library, and Sect. 6 discusses various implementation topics. Section 7 illustrates the utility of ExaMPI. Lastly, Sect. 8 summarizes the efforts toward the ExaMPI library and mentions future work.

2 Background

MPI implementations have existed since 1993, commencing with MPICH [14] (of which the first author of this paper was one of the original authors). Over the past 26 years, MPICH, Open MPI [12], and other open source MPIs have grown in size, complexity, support, and usership. Commercial MPI products based on

proprietary code and/or open source derivatives were also created, and some of them are still in use today.

During this time, there has been consistent and even growing interest in experimenting with MPI in terms of additions, changes, and enhancements to implementations and functionality. Commercial and free derivative products of these open implementations have also been successful, such as Cray MPI [1], Intel MPI [5], IBM Spectrum MPI [2], and MVAPICH [25].

However, the complexity of effecting significant improvements, modifications, and/or changes in design to these large-scale MPIs is a daunting task, with over 1M lines of code present in both middleware products. This large scale makes deep experimentation with or changes to MPI prohibitive, except in device drivers and incremental APIs. For instance, changing the modes of progress or the modes of completion of MPI implementations is a tall order, as is managing their ability to cope with internal concurrency or state.

Furthermore, constrained environments, such as embedded devices and FPGAs, may also prefer to execute MPI functionality without coping with the entirety of large middleware implementations.

In certain of our research projects, we sought to explore different methodologies for implementing MPI, including new modes of strong progress, fault tolerant concepts, and extensions to the standard that require highly effective progress. Such efforts have been thwarted to varying degrees by the legacy assumptions of Open MPI, MPICH, and MVAPICH[1].

The recognition that many MPI applications require only a small to moderate subset of functionality also motivated our design of a new research MPI implementation. By supporting a sound design and allowing functionality to be added systematically over time, we provide incremental ability to run practical codes while reducing the total amount of MPI middleware by orders of magnitude. Furthermore, with a sound, first-principles design, this new MPI would have little dead code, or the technical debt associated with assumptions about node concurrency or progress made in the 1990s.

Furthermore, the availability of high-performance, converged APIs for many networks is a relatively recent development. This growth has enabled ExaMPI's design to focus on Libfabric as the key production networking interface for ExaMPI in addition to fundamental UDP/IP and TCP/IP network drivers. Our decision to focus on Libfabric was pragmatic yet performance-oriented. Libfabric has providers for many fast networks; while Portals is of interest as well, many ideas in Portals have been migrated to Libfabric. UCX was specifically considered but rejected because it is InfiniBand-only, and is immature. However, nothing in our development efforts prevents us or third parties from adding support for Portals and UCX in the future.

[1] In fact, a raft of papers (e.g., [7–10,15,16,18,24,27,29]) in the literature show workarounds to polling progress involving sporadically and haphazardly strewing one's code with MPI_Test. Also, the OSU benchmark for overlap explicitly depends on the use of MPI_Test [4,22].

3 Motivation

The authors' main motivation for MPI is *not* to replace key blocks of functionality or policy in existing production MPI implementations such as MPICH and Open MPI, or in commercial derivatives thereof. Rather, the following motivations have driven the design, implementation, and future roadmap for ExaMPI. The authors intended to

- Enable rapid prototyping of
  - New algorithms for existing MPI operations,
  - New approaches to progress engines and resource allocation in large multicore nodes, and
  - New MPI operations and APIs well before potential standardization;
- Identify and elucidate opportunities to improve MPI at-large, such as
  - Identifying situations where production MPIs could improve performance,
  - Enabling a community of "MPI makers" who can mix and match pieces to build MPI experimental systems and usable middleware in existing and new applications, and
  - Providing enhanced insights into the use of modern programming techniques for "lean middleware" that does not add extra layers or abstraction barriers while being highly maintainable over time;
- Support experimentation, such as
  - Exploration of unified resource management of cores, threads, and memory that unify policies between MPI and OpenMP runtimes and
  - Specialization of entire MPI stacks for large-scale MPI applications that use relatively little of the total source base but want to "aspect" or otherwise tune the MPI specifically for their purposes;
- Support the research interests of the authors, including topics such as
  - Designing, prototyping, and eventually standardizing first-class language interfaces for C++, Python, etc., as opposed to purely transliterations of the current MPI C bindings,
  - Exploration of new fault-tolerant MPI approaches (e.g., MPI Stages [28]),
  - Integration of multiple fault-tolerant MPI models insofar as possible,
  - Provision of first-rate support for persistent collective operations using a an RDMA implementation strategy internally,
  - Creation of a platform for experimentation with MPI Sessions,
  - Demonstration of the value of progress and notification options in an open implementation of MPI (including tunability per application or communicator), formerly only in MPI/Pro [3],
  - Study and prototyping of point-to-point and collective partitioned communication (finepoints) syntax, semantics, and performance, thereby accelerating its path to standardization by the MPI Forum, and
  - Enablement of experiments with MPI in FPGA softcores and in next-generation programmable NICs, such as Mellanox BlueField.

The overall goal is markedly to reduce time from conception to best practices to adoption of new and better MPI through a free, well-designed middleware research platform.

4 Requirements

In developing ExaMPI, we built on specific functional and non-functional requirements. In certain cases, there are quasi-functional requirements related to the modularity and extensibility of the software itself for use in specific applications. The following non-functional requirements were identified at the outset of the project:

1. Use C++ in a modern development style[2]
2. Create understandable code that is maintainable
3. Enable strong progress[3]
4. Enable choice of polling and blocking notification
5. Employ an extensible software architecture that supports extensibility
6. At least support UDP, TCP, and Libfabric transports
7. Enable experimentation with new fault-tolerance models for MPI
8. Focus on persistent and non-blocking operations as fundamental, rather than blocking (point-to-point and collective)
9. Enable a community of contributors of compatible extensions that are BSD-license compatible
10. Enable efficient overlapping of communication and computation when working with high-performance networks

These functional requirements were identified:

1. Support a useful subset of the MPI 3.1 standard[4]
2. Achieve point-to-point throughput that is initially competitive with production free MPI's
3. Achieve latency that is appropriate for a strong progress implementation
4. Specifically, enable the MPI Stages model of MPI fault tolerance [28].

5 Design

In this section, we provide an overview of ExaMPI's design, then discuss progress-engine design, and conclude with ExaMPI's transport design (data movers).

[2] to avoid code cloning, enable use of compiler-supported threads, employ metaprogramming and polymorphism where appropriate, and enable enhanced modularity over C.

[3] independent progress of messages through the network, independently of how often an application calls MPI functions, see also Sect. 5.2.

[4] based on the long experience of the first author and review of many applications' use of MPI as supported by a recent study by some of us and others [23].

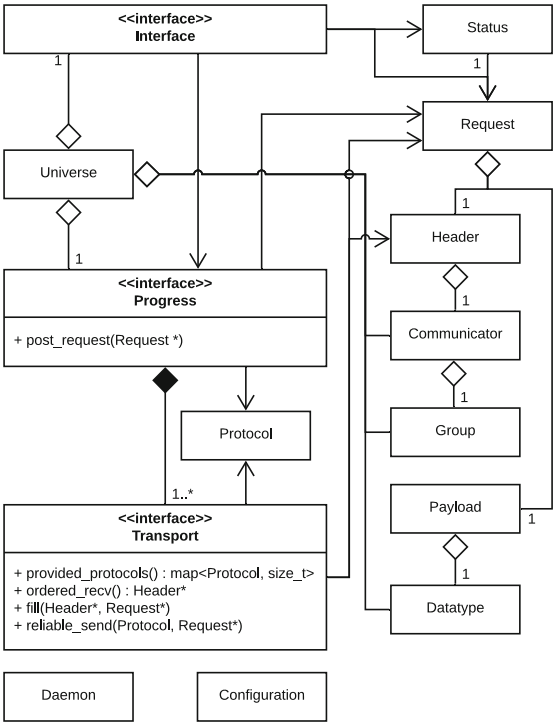


Fig. 1. General overview of the ExaMPI structure

5.1 Overview

The first step in designing ExaMPI was to do a design extraction from the MPI Standard, extending ideas presented in [26]. From this effort thus far, we derived Fig. 1, which provides a Unified Modeling Language (UML) class diagram that also expresses certain choices for practical implementation. This diagram contemplates both the top-down view of the standard APIs and data structures and the bottom-up view of data movers implementing transports for MPI.

Here are some specific facets of the design implied by Fig. 1:

- Different interfaces can be supported for different standard levels (releases) of MPI.
- One can drop in any progress engine rather than having this choice be fixed in the design.
- Transports are also pluggable, akin to other major MPI implementations.
- The Universe is a special class to avoid global state.

Furthermore, the nomenclature and thinking of this MPI's design is required to be compatible with the nomenclature and approach of [6].

Figure 2 reflects the design for the progress engine. First of all, the entire progress engine is pluggable so that other researchers can experiment with alter-

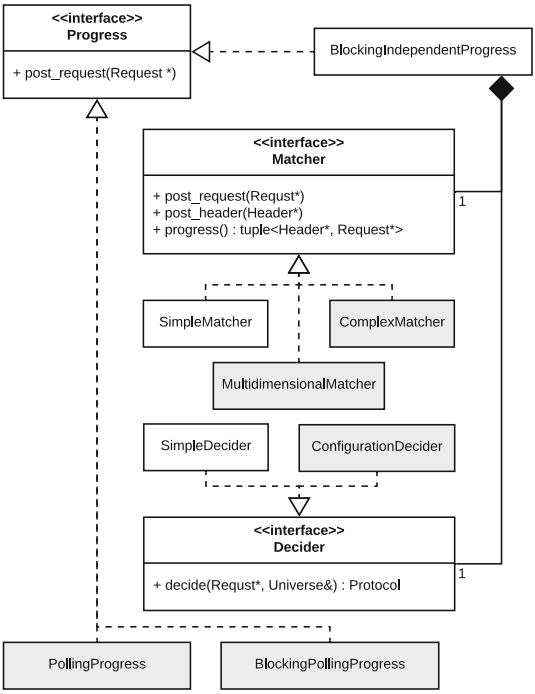


Fig. 2. Structure of the progress layer of ExaMPI

natives to this approach. With reference to Dimitrov's diagram [11] as shown in Fig. 3, we enable all these possible combinations of message progress and notification. This feature is important because prior work has shown that overlapping of communication and computation is severely hampered by polling behavior in progress and/or notification.

5.2 Progress Engine Design

The progress engine abstraction is designed to allow any progress engine to be implemented inside the ExaMPI library. We restrict all progress to be made through the progress engine by requiring all operations to construct a request object. The request object is posted to the progress engine, which then will progress the request objects and the underlying transport implementations.

Figure 3 shows the four classes of MPI progress engines that are possible to be constructed. Currently, ExaMPI implements a strong progress engine. By "strong" we mean that the progress is independent with separate progress threads from the user threads and the notification of completion is blocking. When a user thread waits on a request, the user thread is unscheduled until the request is complete. Further progress engines are being developed to implement the weak and saturated progress classes.

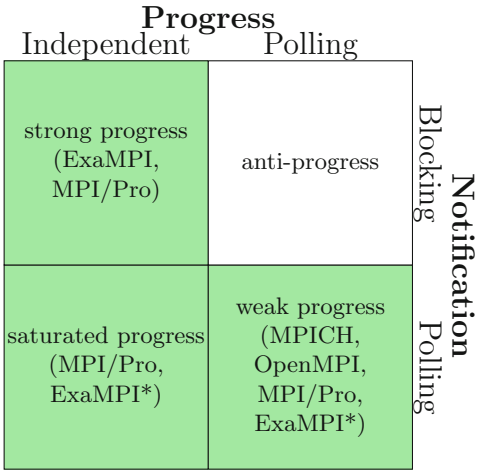


Fig. 3. Dimitrov's progress and notification classification diagram; * Forthcoming modes in ExaMPI

Figure 2 shows the decomposition of the functionality within the Progress class. We separate the matching engine from the progress engine through an interface that allows us to implement many separate algorithms to perform matching. Currently the SimpleMatcher implements a unmatched message queue and posted received queue with a complexity of $O(N^2)$.

In addition, we decompose further the mechanism for decision about which protocol and algorithm is to be used for any MPI operation. The SimpleDecider object implements the expected behaviour of the point-to-point functionality. By implementing a custom decider class, developers can map any MPI operation to any underlying algorithm.

5.3 Transport Design

The transport layer present within ExaMPI is intended to allow abstraction of all available network APIs. Figure 4 shows the hierarchy and required functions any Transport class currently is required to have implemented. Further development on this aspect of the library will enable offloading collectives and one-sided remote memory operations.

The current implementations present are the UDP and TCP transports, which allow for global usage but are not as performant as a high-performance network. Each transport implementation is entirely responsible for handling the memory associated with the network. As such, TCP and UDP use the kernel IP stack as a form of network buffer, but buffer payloads separately once received. Future implementations with more complex communications fabrics will require handling of receive queues.

6 Implementation

In this section, we describe the interface layers (language bindings) and our comprehensive usage of C++17 [21] for performance, flexibility, and code quality.

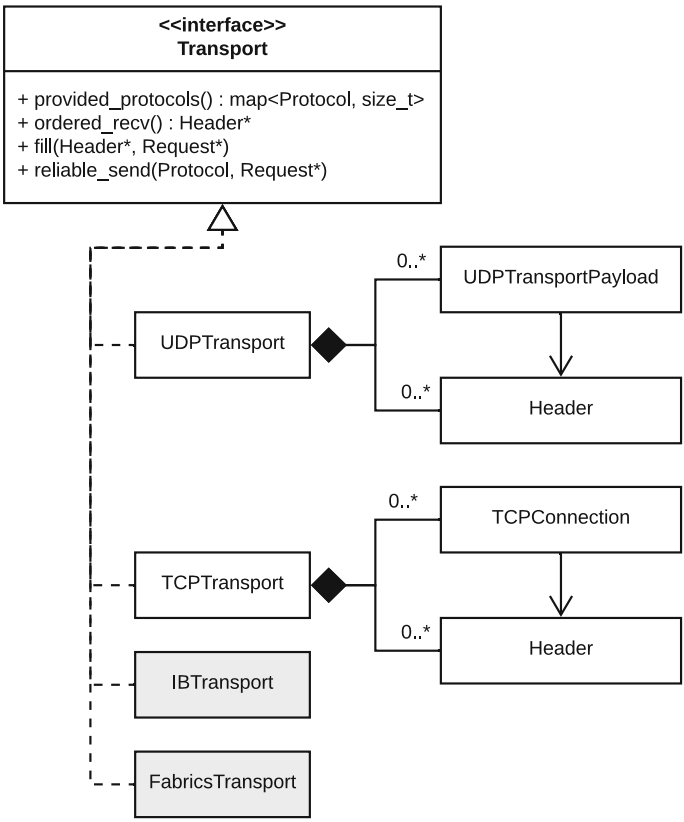


Fig. 4. Structure of the transport layer in ExaMPI

6.1 Interface Layers

The MPI Standard defines bindings for both C and Fortran, which must be available from any compliant MPI implementation. Currently ExaMPI only provides the C language interface, but building the Fortran interface is trivial above the current implementation, similar to other MPI implementations that build their Fortran bindings to simply call the C bindings.

In Fig. 5, the current interface structure of ExaMPI is presented. The C symbol names for both the MPI layer and PMPI layer are defined in the *mpi.h* header file. The MPI symbols are defined to be weak linked to facilitate the

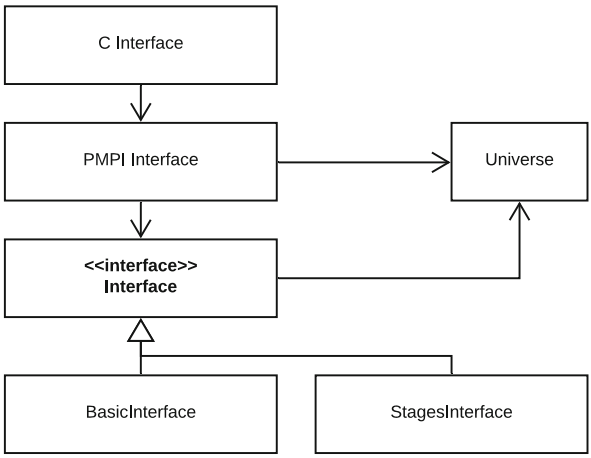


Fig. 5. UML describing interface layers

overloading of their functionality by MPI-compatible tools. The default MPI functions directly call the equivalent PMPI function.

The PMPI layer then uses the root universe to find the interface to the underlying C++ interface. The Interface class declares and defines the same interface as the C bindings of MPI but within a C++ class structure. This structure allows the abstraction of various interfaces for further work such as MPI Stages or FA-MPI.

In addition to extendability, the BasicInterface class allows us to encapsulate all top-level MPI behavior into a single location, which includes error checking and subsetting of blocking and non-blocking paths into persistent path, which is implemented by the underlying library.

6.2 Utilization of C++17

The ExaMPI implementation is written using C++17, which enables many productivity and language features that are not present in earlier C++ or C specifications. In addition, C++ allows for object-oriented programming, which allows the MPI implementation to directly deal with objects instead of the handles to objects. Using objects allows us to develop expressive source code without the clutter required with a C implementation.

We intend for ExaMPI to support full thread safety—with as much internal concurrency as reasonably possible—through the entire library. This arrangement is currently achieved with locks provided by the C++11 specification. By utilizing the built-in threading facilities, we reduce our dependence on external libraries. In the future, we will develop thread-safe, lockless data structures that will allow for the overhead of locking to be removed. They are also supported with built-in atomic operations.

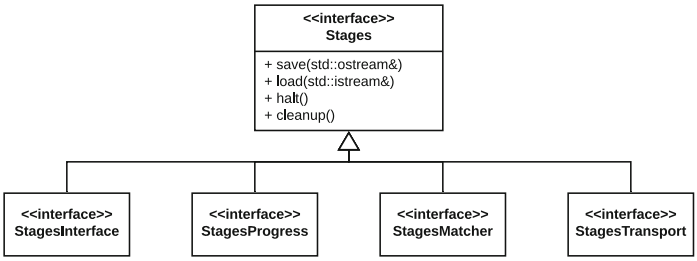


Fig. 6. UML description of stages

The C++ language also provides many capabilities that are tedious to use in C. One of these capabilities is string objects, which provide simpler handling of textual data. Another is exception handling. In C, the error code mechanism requires branches through the code base and forces design decisions. Within C++, exceptions are provided and allow for much cleaner internal working with errors. We utilize exceptions throughout the internal MPI library but provide error codes to the top-level MPI layer.

7 Early Applications of ExaMPI

In this section, we describe practical applications of ExaMPI. Two distinct fault-tolerant MPI models have been integrated into ExaMPI. The prospect of supporting multiple fault-tolerant MPI models at once is also discussed.

7.1 MPI Stages

MPI Stages [28] is a global-restart model that supports fault tolerance in bulk synchronous MPI applications. In this model, a checkpoint of the MPI state is saved along with the application checkpoint: this MPI state can contain a state that is visible to the user (e.g., MPI communicators) and an MPI state that is only visible to the MPI library (such as network connectivity). Each stage is a period between synchronous checkpoints and provides a temporal containment of faults. Upon failure, the runtime system transparently replaces the failed process by restoring both the MPI and application states, respectively, from their last synchronous checkpoints and continue without restarting the overall MPI job. Live processes roll back only a few iterations within the main loop instead of rolling back to the beginning of the program, while a replacement of a failed process restarts and reintegrates, thereby achieving faster failure recovery.

We introduced failure recovery support using MPI Stages in ExaMPI. The ExaMPI library isolates the new API extensions (StagesBasicInterface) added by MPI Stages from the standard MPI API as shown in Fig. 5. We can enable the MPI Stages feature in ExaMPI library using the command line parameter – *enable_mpi_stages* with mpiexec. The two main design aspects of MPI Stages are

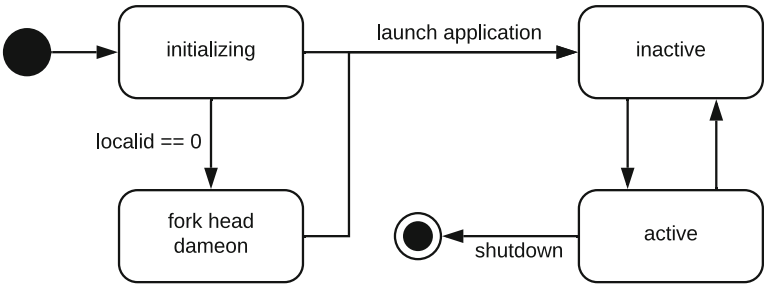


Fig. 7. State diagram of fault dæmon

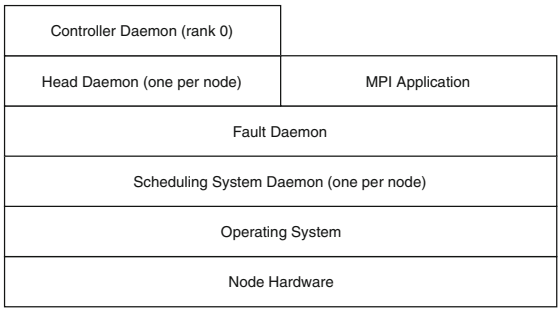


Fig. 8. Runtime system dæmon layers

(1) managing and capturing internal MPI state and (2) transparent replacement of a failed process by the MPI runtime system.

MPI Stages requires the user to save and load the internal state of MPI. To support state checkpointing, the ExaMPI library provides a `Stages` interface as shown in Fig. 6. Different modules of the ExaMPI library (e.g., Progress, Transport) could save/load their MPI state in a checkpoint by implementing this interface. The interface also provides a halt and cleanup method to handle process failure for live processes.

The ExaMPI runtime system supports the transparent replacement of a failed process as required by MPI Stages. The runtime system of ExaMPI supports both local execution and the Slurm job scheduling system. The runtime is designed using a double-dæmon method for compatibility with many job scheduling systems. The job scheduling system has the management daemon on the node that executes the second layer dæmon, the fault dæmon. The fault dæmon then in turn launches the MPI application as a sub-process. This design allows us to intercept any failure of the MPI application and prevents the job scheduling system from terminating the job prematurely. The fault dæmon also prepares the final application environment variables. The fault dæmon waits for the process to terminate and thereby avoids using any processing power from the node.

```
int main()
{
    /*MPI Initialization and variable setup*/
    while(operations_left)
    {   /*Start the FA-MPI Transaction*/
        MPI_TryBlock_start(comm, MPI_TRYBLOCK_GLOBAL, &try_request);
        /*Do normal MPI Operations*/
        MPI_Operation();
        /*Finish the FA-MPI Transaction*/
        return_code = MPI_TryBlock_finish(try_request);
        /*Check for errors in the transaction*/
        if(MPI_SUCCESS != return_Code)
        { /*Perform recovery*/ }
    }
    /*MPI Finalize and other cleanup*/
}
```

Fig. 9. A simple example of an FA-MPI TryBlock

This feature is useful because there is a one-to-one application and fault dæmon relationship. Figure 7 shows the state diagram of the fault dæmon.

Potentially, two more dæmons are launched. First the fault dæmon, which both executes the local root rank and executes the head dæmon on each node. The head dæmon launches the final dæmon, the controller dæmon, if it is rank 0. With this structure, there are two dæmons (head dæmon and controller dæmon) awake on the root node of the job allocation and one dæmon (head dæmon) on every other node. This arrangement enables hierarchical scaling for side-channel communication—that is, data not directly related to the MPI application via socket-based TCP/IP. Figure 8 shows the sub-process relationships.

7.2 FA-MPI

Fault-Aware MPI (FA-MPI) is a lightweight, transaction-based fault tolerance model for MPI [17]. With this transaction-based model, an application can choose to use FA-MPI to achieve a fine-grain fault tolerance model by encapsulating every MPI operation in a single transaction. Or, should the application want to balance performance with fault tolerance, the application can choose to instead put many MPI operations into a single transaction. As FA-MPI is designed to be an extension of the MPI API, the application can use the fault awareness provided by FA-MPI to determine the level of fault tolerance it wants. While FA-MPI does provide the means to do failure recovery, it is also flexible and lets the application decide what its failure recovery method should be. As stated in the original documentation, "Applications using FA-MPI will run to completion with higher probability than with a non-fault aware MPI" [17].

These nestable transactions allow a series of operations to be committed if they were successful or to be retried if a fault was detected by one or more pro-

cesses. To create these transactions, FA-MPI uses non-blocking collective functions and encapsulates a transaction with special TryBlock functions. TryBlock transactions can be local in scope—where only the local process must decide what to do—or global in scope—where all failures are synchronized among all processes. FA-MPI incorporates timeouts into its fault detection methods to prevent the possibility of deadlock introduced when dealing with fault tolerance. These timeouts can be user defined. FA-MPI also allows users to raise their own errors, should users want to test their fault tolerance model or have a fault that MPI can't detect. Once a TryBlock has finished, all alive processes will have the same view of the current status of the program. Processes then query FA-MPI for information about the fault, which can then be used to form a consensus on how to proceed. Figure 9 provides a brief example of how the TryBlocks might be used. To help with the recovery process, FA-MPI provides functionality to repair or rebuild communicators that were potentially broken during the fault.

FA-MPI is in the process of being integrated into ExaMPI. This integration will provide several benefits to both ExaMPI and FA-MPI. The first noticeable benefit for FA-MPI is ExaMPI's fundamental focus on non-blocking operations, something FA-MPI makes heavy use of and requires from MPI for its implementation [17]. Additionally, ExaMPI allows for further exploration of the FA-MPI fault tolerance model and how the model performs in the presence of faults. Since FA-MPI has already been implemented and tested in OpenMPI, we have a base level to compare how well FA-MPI performs in ExaMPI. Lastly, ExaMPI will also help quantify the necessary functionality required from MPI implementations to support its fault tolerance model, which will help enable ExaMPI (and perhaps other MPIs) to support fault tolerance models more successfully.

7.3 Multi-FT Library

The authors are designing approaches to compose multiple fault-tolerant models into ExaMPI. Apart from single models such as Reinit or ULFM and checkpoint-restart, we are not aware of successful integration of multiple models. Both the syntax and semantics of such combined models are of interest, but implications for MPI middleware architecture are also of tremendous consequence. We are exploring how to manage the complementary, at-times conflicting, and otherwise independent impacts on an MPI implementation arising from multiple models, including how to manage conflict resolutions between multiple models.

8 Conclusions and Future Work

In this paper, we described ExaMPI, a new, experimental implementation of the MPI Standard. ExaMPI solves the problem that full-scale open source MPIs are legacy middleware projects of large-scale and long-running development by many contributors; they are difficult to learn, modify, and use for middleware research, except in limited ways. Where they are usable, they are adequate, but many

experiments are either intractable or require students and professors to spend inordinate amounts of time "modifying around the edges" of such middleware.

Thus far, ExaMPI has proven to be a useful research vehicle for a small number of people. As we move to a community of developers, researchers, and users, we look to increasing that utilization dramatically and expect the modularity of design to allow for many interesting hybridizations of our baseline code and concepts with others' ideas, prototypes, and additions.

A set of robust additions and experiments with ExaMPI are planned for the near future and include support for MPI-4 persistent collective communication [20], support for finepoints [13], and support for MPI-4 Sessions [19]. Also, revisiting unrolling of collective operations and experiments for communication and computation overlap are planned. These projects and others are keyed to the research interests of the authors and their immediate collaborators. We expect further research undertaken by others once the fall 2019 release of ExaMPI occurs contemporaneously with the publication of this paper.

References

1. Cray MPI. https://pubs.cray.com/content/S-2529/17.05/xctm-series-programming-environment-user-guide-1705-s-2529/mpt
2. IBM Spectrum MPI. https://tinyurl.com/yy9cwm4p
3. MPI/Pro. https://www.runtimecomputing.com/products/mpipro/
4. Osu micro-benchmarks 5.6.2. http://mvapich.cse.ohio-state.edu/benchmarks/
5. Intel MPI library, August 2018. https://software.intel.com/en-us/mpi-library
6. Bangalore, P., Rabenseifner, R., Holmes, D., Jaeger, J., Mercier, G., Blaas-Schenner, C., Skjellum, A.: Exposition, clarification, and expansion of MPI semantic terms and conventions (2019). Under review
7. Barigou, Y., Venkatesan, V., Gabriel, E.: Auto-tuning non-blocking collective communication operations. In: 2015 IEEE International Parallel and Distributed Processing Symposium Workshop, pp. 1204–1213, May 2015. https://doi.org/10.1109/IPDPSW.2015.15
8. Castillo, E., et al.: Optimizing computation-communication overlap in asynchronous task-based programs: poster. In: Proceedings of the 24th Symposium on Principles and Practice of Parallel Programming, PPoPP 2019, pp. 415–416. ACM, New York (2019). https://doi.org/10.1145/3293883.3295720
9. Denis, A., Trahay, F.: MPI overlap: benchmark and analysis. In: 2016 45th International Conference on Parallel Processing (ICPP), pp. 258–267, August 2016. https://doi.org/10.1109/ICPP.2016.37
10. Didelot, S., Carribault, P., Pérache, M., Jalby, W.: Improving MPI communication overlap with collaborative polling. In: Träff, J.L., Benkner, S., Dongarra, J.J. (eds.) EuroMPI 2012. LNCS, vol. 7490, pp. 37–46. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-33518-1_9
11. Dimitrov, R.P.: Overlapping of communication and computation and early binding: fundamental mechanisms for improving parallel performance on clusters of workstations. Ph.D. thesis, Mississippi State, MS, USA (2001)
12. Graham, R.L., Shipman, G.M., Barrett, B.W., Castain, R.H., Bosilca, G., Lumsdaine, A.: Open MPI: a high-performance, heterogeneous MPI. In: Cluster 2006, pp. 1–9, September 2006

13. Grant, R.E., Dosanjh, M.G.F., Levenhagen, M.J., Brightwell, R., Skjellum, A.: Finepoints: partitioned multithreaded MPI communication. In: Weiland, M., Juckeland, G., Trinitis, C., Sadayappan, P. (eds.) ISC High Performance 2019. LNCS, vol. 11501, pp. 330–350. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-20656-7_17
14. Gropp, W.: MPICH2: a new start for MPI implementations. In: Kranzlmüller, D., Volkert, J., Kacsuk, P., Dongarra, J. (eds.) EuroPVM/MPI 2002. LNCS, vol. 2474, pp. 7–7. Springer, Heidelberg (2002). https://doi.org/10.1007/3-540-45825-5_5
15. Guo, J., Yi, Q., Meng, J., Zhang, J., Balaji, P.: Compiler-assisted overlapping of communication and computation in MPI applications. In: 2016 IEEE International Conference on Cluster Computing (CLUSTER), pp. 60–69, September 2016. https://doi.org/10.1109/CLUSTER.2016.62
16. Hager, G., Schubert, G., Wellein, G.: Prospects for truly asynchronous communication with pure MPI and hybrid MPI/OpenMP on current supercomputing platforms (2011)
17. Hassani, A.: Toward a scalable, transactional, fault-tolerant message passing interface for petascale and exascale machines. Ph.D. thesis, UAB (2016)
18. Hoefler, T., Lumsdaine, A.: Message progression in parallel computing - to thread or not to thread? In: 2008 IEEE International Conference on Cluster Computing, pp. 213–222, September 2008. https://doi.org/10.1109/CLUSTR.2008.4663774
19. Holmes, D., et al.: MPI sessions: leveraging runtime infrastructure to increase scalability of applications at exascale. In: EuroMPI 2016, pp. 121–129. ACM, New York (2016)
20. Holmes, D.J., Morgan, B., Skjellum, A., Bangalore, P.V., Sridharan, S.: Planning for performance: Enhancing achievable performance for MPI through persistent collective operations. PARCOMP **81**, 32–57 (2019)
21. ISO: ISO/IEC 14882:2017 Information technology – Programming languages – C++. Fifth edn., December 2017. https://tinyurl.com/yct5hxcs
22. Liu, J., et al.: Performance comparison of MPI implementations over Infiniband, Myrinet and Quadrics. In: Proceedings of the 2003 ACM/IEEE Conference on Supercomputing, SC 2003, pp. 58–58, November 2003. https://doi.org/10.1109/SC.2003.10007
23. Laguna, I., Mohror, K., Sultana, N., Rüfenacht, M., Marshall, R., Skjellum, A.: A large-scale study of MPI usage in open-source HPC applications. In: Proceedings of the SC 2019, November 2019 (2019, in press). https://github.com/LLNL/MPI-Usage
24. Lu, H., Seo, S., Balaji, P.: MPI+ULT: overlapping communication and computation with user-level threads. In: 2015 IEEE 17th International Conference on High Performance Computing and Communications, 2015 IEEE 7th International Symposium on Cyberspace Safety and Security, and 2015 IEEE 12th International Conference on Embedded Software and Systems, pp. 444–454, August 2015. https://doi.org/10.1109/HPCC-CSS-ICESS.2015.82
25. Panda, D.K., Tomko, K., Schulz, K., Majumdar, A.: The MVAPICH project: evolution and sustainability of an open source production quality MPI library for HPC. In: WSPPE (2013)
26. Skjellum, A., et al.: Object-oriented analysis and design of the message passing interface. Concurrency Comput.: Practice Exp. **13**(4), 245–292 (2001). https://doi.org/10.1002/cpe.556

27. Sridharan, S., Dinan, J., Kalamkar, D.D.: Enabling efficient multithreaded MPI communication through a library-based implementation of MPI endpoints. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2014, pp. 487–498. IEEE Press, Piscataway (2014). https://doi.org/10.1109/SC.2014.45
28. Sultana, N., Rüfenacht, M., Skjellum, A., Laguna, I., Mohror, K.: Failure recovery for bulk synchronous applications with MPI stages. PARCOMP **84**, 1–14 (2019)
29. Wittmann, M., Hager, G., Zeiser, T., Wellein, G.: Asynchronous MPI for the masses. arXiv preprint arXiv:1302.4280 (2013)